

Classifier-Guided Autoencoder Pretraining

Abstract: In this document, we study the effectiveness of using pretrained image classifiers to guide the learning of autoencoder representations for downstream image classification. Using the CIFAR-10 dataset and a pretrained ResNet model from Y. Chen [1], we train convolutional autoencoders on a joint loss function combining reconstruction loss and the classifier loss on the reconstructed outputs. We then evaluate the performance of classifier models constructed by attaching classifier heads to the trained encoder.

Declaration: The code for this project was written entirely by myself with the assistance of GPT-5.5. My approach to this project was largely guided by what I have learnt from the course *A deep understanding of deep learning*, taught by Mike X Cohen. No part of this document was written with GPT-5.5 or any other LLM.

Motivation

It is well-known that autoencoders learn compressed representations of images in their latent layer, but these representations are trained using pure reconstruction loss which sometimes obfuscates features in the image which are relevant to classifiers. I wanted to explore if training autoencoders to reconstruct images faithfully while also preserving classifier performance allows the autoencoder to learn representations which are more useful for classification. To measure effects in this direction, my plan was to compare the performance of models trained with the approach outlined in the abstract with the performance of more typical CNN classifiers trained from scratch. In particular, I was curious to find out if this could be used for the same purpose as distillation – training a simpler classifier with comparable performance to the pretrained classifier.

I chose the CIFAR-10 dataset for its relatively small size, which made it ideal for iterative development, and because it is difficult to saturate with relatively simple models (unlike e.g. MNIST), which made it easier to study if my approach was having any effect. I did have concerns that the low initial resolution of CIFAR-10 images might make it difficult to achieve meaningful compression with an autoencoder. I would be interested to adapt my project to use STL-10 instead.

Setup

I downloaded the CIFAR-10 dataset from the torchvision library and imported my choice for the pretrained model from [chenyaofu/pytorch-cifar-models](https://github.com/chenyaofu/pytorch-cifar-models), a ResNet-20 model. The only transform I used for the images was `T.ToTensor()` as I decided it would be best for the autoencoder to reconstruct the original image; the normalisation required for the ResNet-20 was included as a preprocessing step in the definition of the classifier model I used. Finally, since the pretrained model was trained on the training split, the devset and test set were randomly selected from the test split.

Experiment

First, I trained a naive autoencoder purely on reconstruction loss to get a baseline indication of how the pretrained classifier performs on reconstructed output. The encoder network had three convolutional blocks, consisting of convolutional layers with 4 by 4 kernels, padding 1 and stride 2, each followed by ReLU activation layers. The number of channels was $3 \rightarrow 16 \rightarrow 32 \rightarrow \text{latent_channels}$ in the latent layer. The decoder network mirrored the encoder. With `latent_channels = 64`, this gave a 1024-dimensional latent representation ($64 \times 4 \times 4$), compared to the 3072-dimensional input ($3 \times 32 \times 32$). The autoencoder was trained for 50 epochs.¹

It was straightforward to verify the visual similarity of the reconstructions, but I also wanted to know how it would shift the classifier's output distribution; for this, I used the following measures:

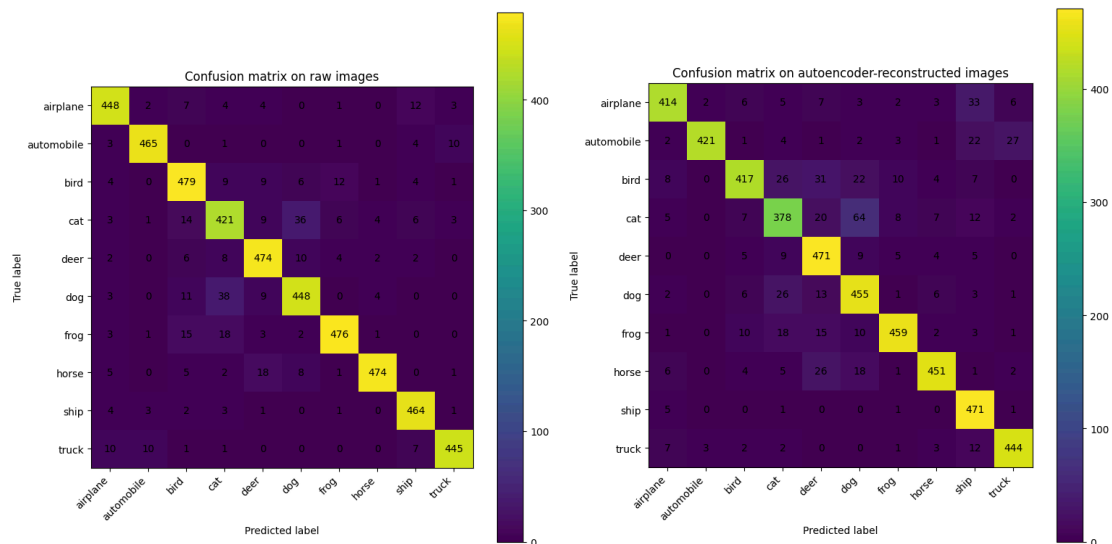
- **classification accuracy** – the accuracy of the classifier on the reconstructed output
- **classifier loss** – the cross-entropy loss of the classifier on the reconstructed output
- **classifier consistency** – the proportion of images where the classifier returned the same classification for the original image and the reconstructed output
- **KL divergence** – the KL divergence between the logit distributions of the classifier on the original image and on the reconstructed output

as well as analysis of which categories had the worst performance.

For this autoencoder model, evaluation on the devset gave classification accuracy of 87.6%; classifier loss of 0.4796; classifier consistency of 90.6%; and KL divergence of 0.2392. The reconstruction loss was 0.00045.

(Please see the next page.)

¹ I initially tried an autoencoder network with a $16 \times 4 \times 4$ latent layer, but this only achieved 42% classification accuracy with the frozen classifier.



The figure on the left is a confusion matrix for the pretrained classifier on the raw images. We can see that the model performs well on all categories, with some difficulty distinguishing between cats and dogs. On the right is the confusion matrix for the same classifier on reconstructed images. We observe that the autoencoder accentuates the difficulty of distinguishing between animals, especially for cats and birds, and between vehicles.

Notably, the classifier is significantly less likely to predict that the image depicts a bird on reconstructed outputs (458 predictions of bird on reconstructed output vs. 540 predictions on the original data). The bird category saw the worst degradation in performance, with accuracy decreasing from 91% to 79%.

I experimented with some modifications to the hyperparameters for the naive autoencoder architecture described above to see if I could achieve better performance without further increasing the dimension of the latent space, such as using an L1 term in the reconstruction loss, but I did not find any changes which meaningfully improved the performance. I decided to proceed with training a classifier-guided autoencoder rather than spend too much time optimising the hyperparameters, but this is something which I would like to revisit.

The first attempt to train the classifier-guided model did not work; it resulted in a model with a much higher reconstruction loss of 0.032 and a poor KL divergence of 0.5943. This was partially due to the relative sizes of the gradients for the reconstruction loss and the classifier loss. To try to fix this, I separated the epochs into blocks with different weightings for the joint loss and dramatically reduced both the weighting of the classifier loss and the learning rate across all of the blocks. I also implemented a scheduling function to make the shift from pure reconstruction loss to the joint loss more gradual within blocks and tested other regularisation techniques for the training.

To my surprise, none of the methods I tried resulted in performance significantly exceeding that of the naive autoencoder, despite the much longer time I spent iterating with the devset. During this process, I also realised that I had made a mistake by not fixing the weight

initialisation for the training runs, which would detract from the reliability of a positive result (since any improvements could potentially be due to fortuitous weight initialisation); however, it supports the null result.

Therefore, my conclusion for this experiment is that adjusting the loss function by adding a term to preserve the performance of a pretrained classifier is not a viable approach for training autoencoders to guide new classifiers.

By observing the model performance statistics for each epoch during training, I developed some intuitions for the geometry of the loss landscape. I noted that:

1. training on pure reconstruction loss generally decreases classifier loss – as also demonstrated by the naive autoencoder
2. training with the joint loss sometimes increases reconstruction loss
3. training with joint loss first reduced both reconstruction loss and classifier loss, but later switching to pure reconstruction loss often **dramatically increased classifier loss**, at least for the first few epochs

This suggested to me that the loss landscape looks somewhat like the visualisation below:



Image generated with GPT-5.5.

where darker blue represents lower reconstruction loss and darker purple represents lower classifier loss. There is a general trend towards darker blue and a higher density of purple dots approaching the top-left corner, but there are purple dots scattered everywhere. This intuitively explains the points above since then:

1. training on pure reconstruction loss decreases classifier loss as it moves the model closer to the top left where there are more purple dots
2. training with the joint loss sometimes increases reconstruction loss if we move away from the top left towards a nearby purple dot

3. switching to pure reconstruction loss after first training with a joint loss dramatically increases classifier loss for a few epochs because we first move to one of the dots closer to the bottom right, then we start moving towards the top left and temporarily are distanced from any purple dots

This seems like an unusually persuasive theory given its simplicity (a theory which represents the loss landscape as a 2-dimensional space rather than a much higher dimensional space) and it is one that I intend to investigate further.

Experiment – Part 2

I decided to briefly investigate the difference in performance between two models consisting of an encoder with a classifier head if one model was trained from scratch while for the other, only the classifier head was trained, with the help of a frozen pretrained encoder. The code used for the classifier head is below:

```
self.classifier = nn.Sequential(  
    nn.AdaptiveAvgPool2d((1, 1)),  
    nn.Flatten(),  
    nn.Linear(latent_channels, 64),  
    nn.ReLU(),  
    nn.Dropout(dropout),  
    nn.Linear(64, num_classes)  
)
```

while the structure of the encoder was the same as for the autoencoder described earlier.

This was not intended to be a rigorous experiment, though I would be keen to develop it into one with more time. The preliminary results I found were that the model trained from scratch significantly outperformed the model trained with a frozen encoder network, with a final (devset) accuracy of 64.0% compared to 38.9%. However, both models showed signs of overfitting judging by the difference in train loss and dev loss, so there may be a lot of room for improvement for both even with the same limited architecture.

Improvements and limitations

In hindsight, I should have been more diligent in noting down and saving the choices of hyperparameters which led to sub-par performance, so that I could contrast them more easily with my final results. This applies in particular to the weight initialisations, which I should have controlled as I noted earlier. There are also some methods which I simply didn't try here, such as batch normalisation. Lastly, I could have been more flexible with my training procedure to waste less GPU time per iteration and generally accelerate my research – for example, by writing better pipelines for training and testing models.

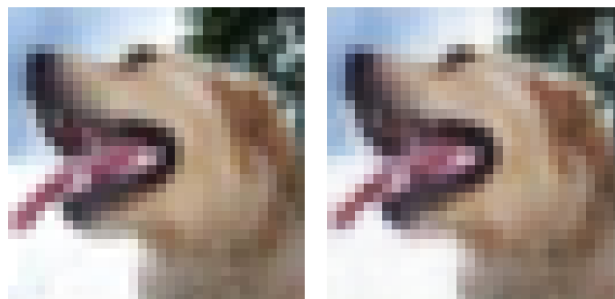
The experiment is limited by being restricted to only CIFAR-10, and the specific (and largely arbitrary) choices of the teacher model, autoencoder architecture and classifier head architecture, all of which could be varied to expand the search for results. The work would also undoubtedly benefit from comparison to papers which have researched similar ideas such as feature matching, which was not considered here.

There is another more subtle limitation of my approach, which I realised from the start, but I was unsure about how to avoid it. My experiments involved training a model with a fixed architecture, then modifying that model and comparing the results on evaluation. The issue with my approach is that the models are iterated on the devset separately with different techniques. This introduces a source of random error if my iteration unpredictably produces better results for one model than the other, independently of the architecture, or even systematic error due to the choices of architecture – for example, if the architecture of one model makes it much easier to find the best hyperparameters than for the other.

Conclusions

The results suggest that pure reconstruction loss is very effective for training autoencoders which allow high pretrained classifier performance on reconstructions – but classifier-guided training with joint loss functions does not improve encoder performance, even as evaluated by the same classifier. The author proposes that this may be due to a specific geometry of the joint loss landscape, but more investigation is needed. The results of this article are limited in scope due to a narrow field of inquiry and some methodological weaknesses.

Example: original image (left) and its reconstruction (right) by an instance of the naive autoencoder.



References

- [1] Yaofo Chen, *PyTorch CIFAR Models*
<https://github.com/chenyaofo/pytorch-cifar-models>